OOAD

Objects, etc.

What is an Object

- An Object is a representation of an entity.
- It can represent something concrete, such as Harsha’s truck or concept such as a bank transaction or a purchase order.
- An Object is an instance of a class.

Identifying Object

- Collect User Stories, Use Case Narratives or any other written requirements.
- Go through them and pick out the nouns.

Eg. Purchase Items

- Customer confirms items in the Shopping Cart
- Customer provides payment and *address* to process sales
- System validate payment Information and respond with confirmation of order and provides order number that customer can use to check on order status.
- System will send confirmation of order details to customer in an email.

Identifying Object

- Go through and pick out the nouns.

Eg. Purchase Items

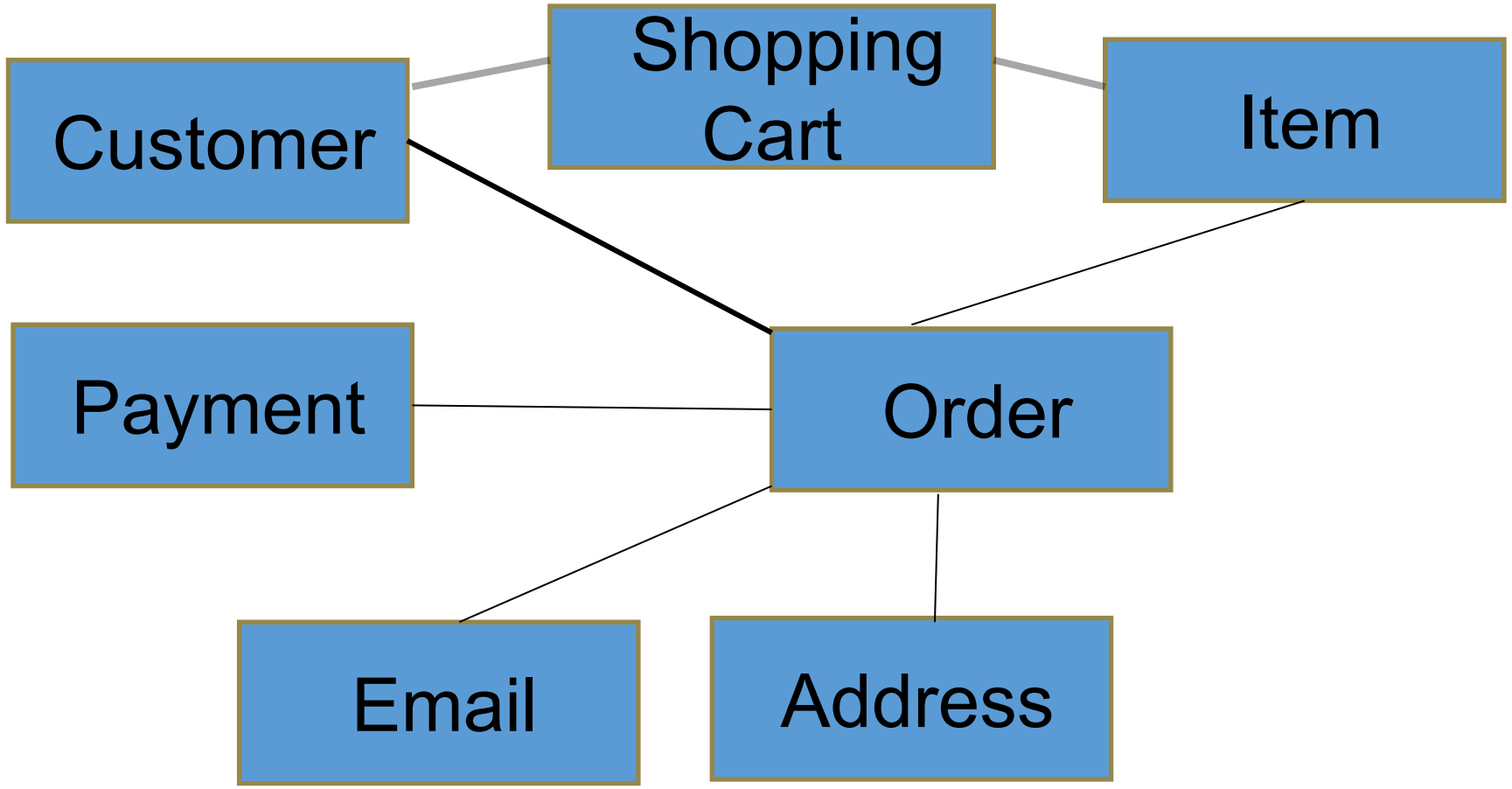
- Customer confirms items in the Shopping Cart
- Customer provides payment and *address* to process sales
- System validate payment Information and respond with confirmation of order and provides order number that customer can use to check on order status.
- System will send confirmation of order details to customer in an email.

Identifying Object

Customer
Shopping Cart
item
payment , *address*
Sales , System
order
order number
order status
order details
email

Filter the Objects

Conceptual Object Model



Identifying Object Responsibilities using verbs or verb phrases

Eg. Purchase Items

- Customer confirms items in the Shopping Cart
- Customer provides payment and *address* to process sales
- System validate payment Information and respond by confirmation of order and provides order number that the customer can use to check on order status
- System will send confirmation of order details to customer in an email.

Identifying Object Responsibilities using verbs or verb phrases

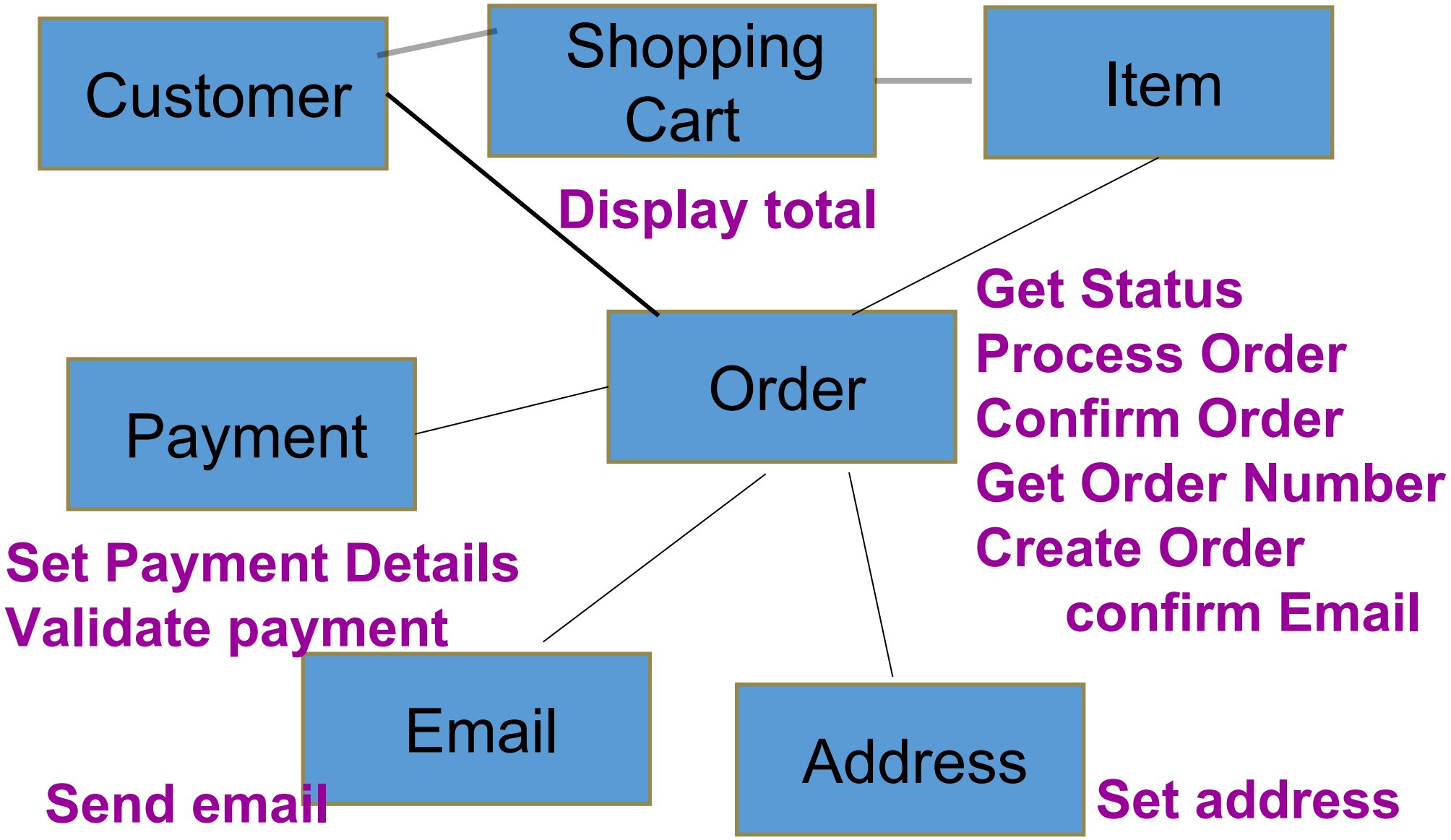
- Not all will become behaviours,
- Some will be combined,
- Some will be split apart,
- Some will not be needed for the system.

Identifying Object Responsibilities using verbs or verb phrases

Eg. Purchase Items

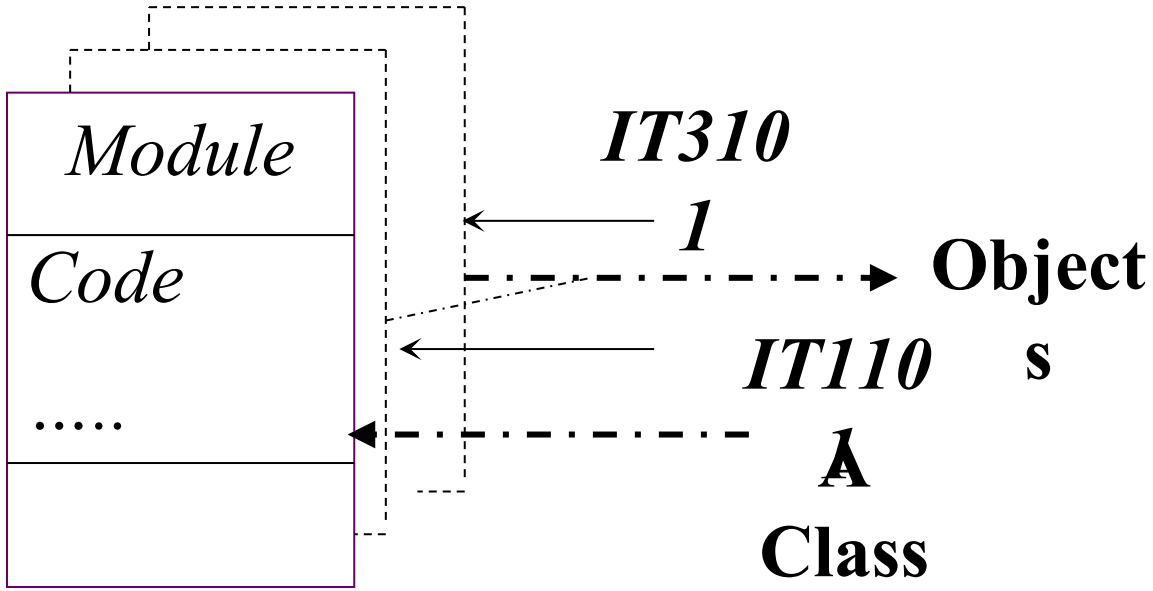
- Customer confirms items in the Shopping Cart
- Customer provides payment and *address* to process sales
- System validate payment Information and respond by confirmation of order and provides order number that the customer can use to check on order status
- System will send confirmation of order details to customer in an email.

Conceptual Object Model



Identifying Object Responsibilities using verbs or verb phrases

- A Class is a description of a group of objects with common properties (attributes), common behavior (operations), common relationships to other objects, and common semantics.



Naming Convention

Almost universally followed

- Name the **Class** in UpperCamelCase

Eg BankAccount

whereas **Objects** are names in lowerCamelCase

Eg. kamalsAccount

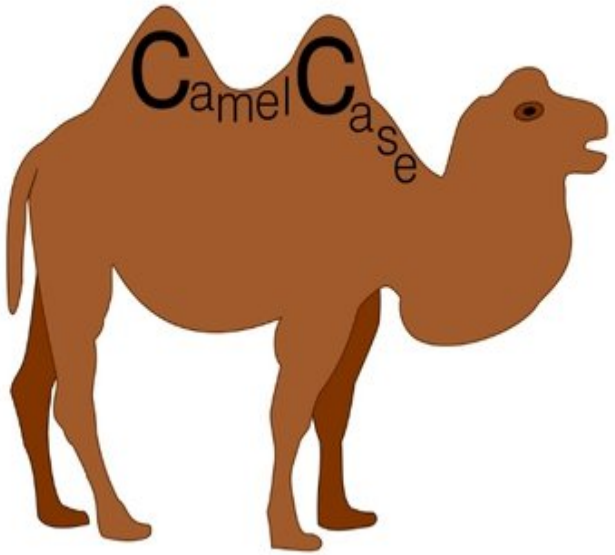
- Avoid abbreviation at all cost

Eg. BnkAct **X**

What is CamelCase?

When spaces are not allowed.

Makes it easier to read.



Visibility – UML notations

- + Public
- - Private
- # Protected
- ~ Package - Any element that is in the same package as the class, or in a nested sub package , can access any of its features with package visibility.
- Implementation languages may differently interpret UML visibility types.
- For general modeling above type are quite adequate.

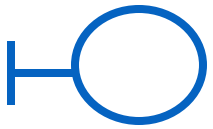
Stereotypes and Classes

A Stereotype is a mechanism you can use to categorize your classes.

- Say you want to quickly find all of the forms in the model,
- You could create a stereotype called *form*, and assign all of your windows this stereotype.
- To find your forms later, you would just need to look for the classes with that stereotype.

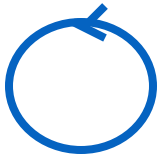
Stereotypes and Classes

- There are *three* primary class stereotypes in UML.



Boundary

Entity



Control

Stereotypes and Classes

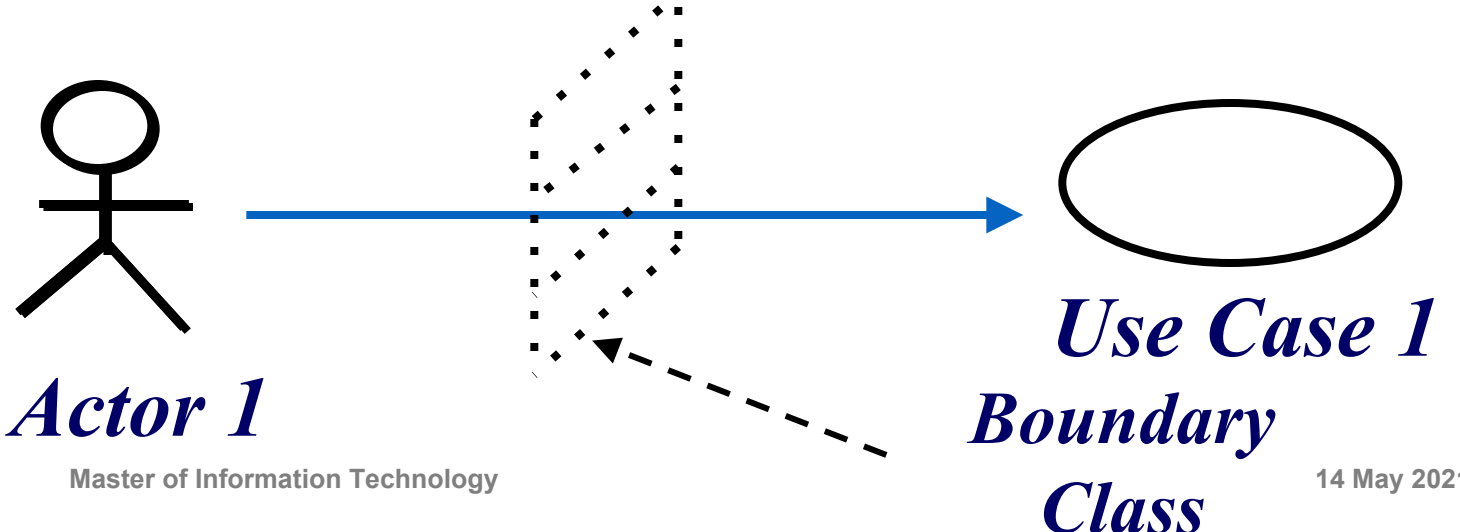
***Boundary** Class:*

- They provide the interface to a user or another system. (ie. Interface to an actor).
- Handles communication between system surroundings and the inside of the system.
- To find the *Boundary* classes, you can examine your Use Case diagram,

Stereotypes and Classes

***Boundary** Class:*

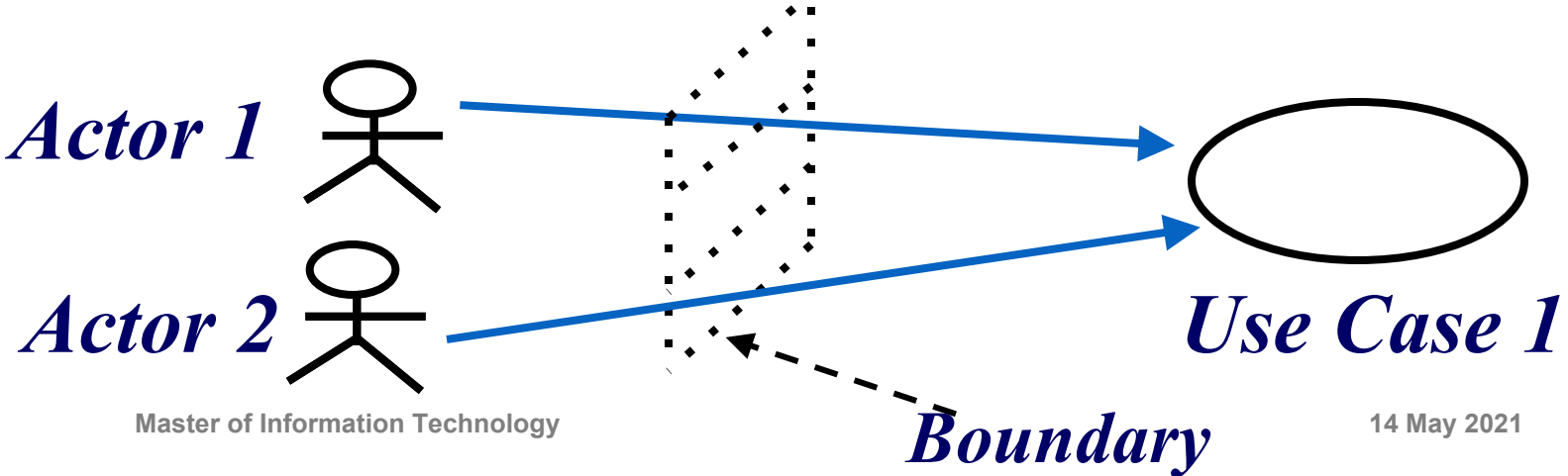
- At a minimum there must be, one *Boundary* class for every actor-use case interaction.
- Boundary class allows actor to interact with the system.



Stereotypes and Classes

***Boundary** Class:*

- You do not necessarily have to create a unique *Boundary* class for every actor-use case pair.
- Two actors may initiate the same use case.
- They might both use the same Boundary class to communicate with the system.



Stereotypes and Classes

- These are classes that mediate between the subject (System boundary) and its environment.
  - User Interface class – classes that interface between the system and humans;
  - System Interface class – classes that interface with other systems;
  - Device Interface class – classes that interface with external devices such as sensors;

Stereotypes and Classes

Entity Class

- They are needed to perform task internal to the system. Reflect a real world entity.

Identifying Entity Classes

Noun/Verb Analysis

- Identify the nouns and noun phrases

Stereotypes and Classes

Entity Class

- The initial list of nouns must be filtered because,
  - it could contain nouns that are outside the problem domain.
  - nouns that are just language expressions.
  - nouns that are redundant.
  - nouns that are attributes.

Control Classes

- A control class is a specialized component, often used in software architecture, UML modeling, or research, designed to manage, coordinate, and control interactions between other objects, data, and user interface elements.
- It acts as a mediator, handling application logic and ensuring that different system components work together effectively.

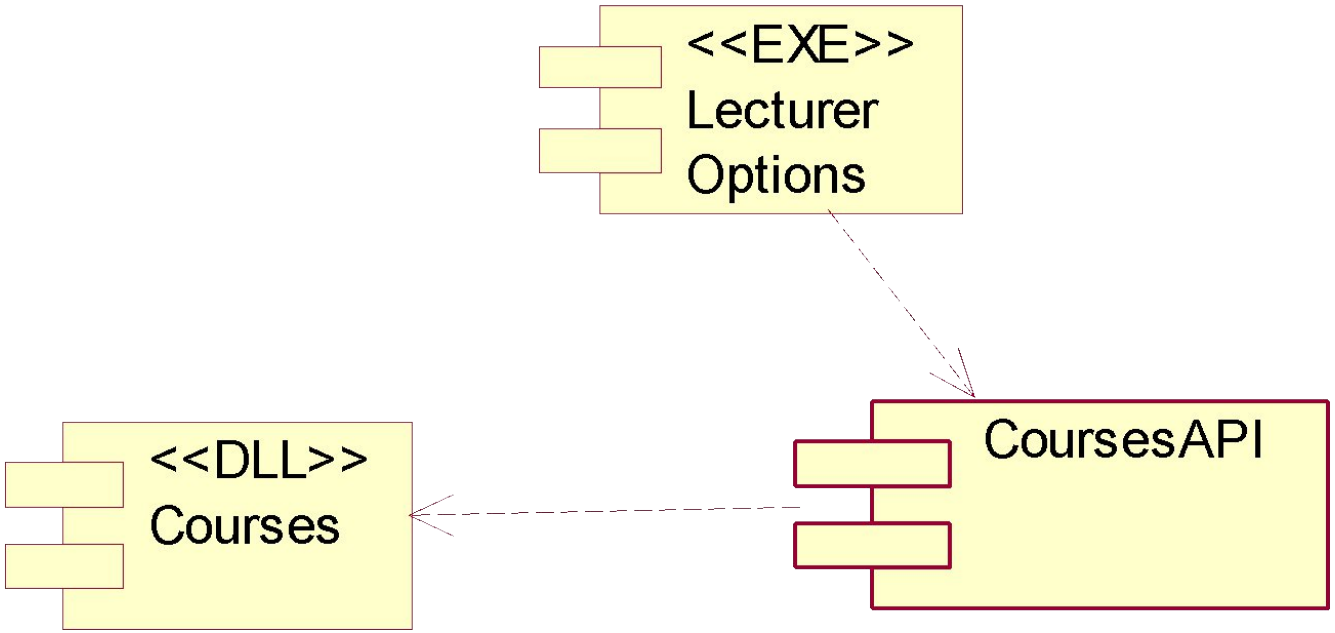
Component diagram

- Modern software development proceeds via components (eg. Table, data file, executable, DLL, document etc) which is particularly important in team-based development efforts.
- Why do you model components:
  - Clients can see the structure of the finished system
  - Developers have a structure to work toward
  - Technical writers who have to provide documentation and help files can understand what they are writing about
  - You are ready for reuse

Component diagram

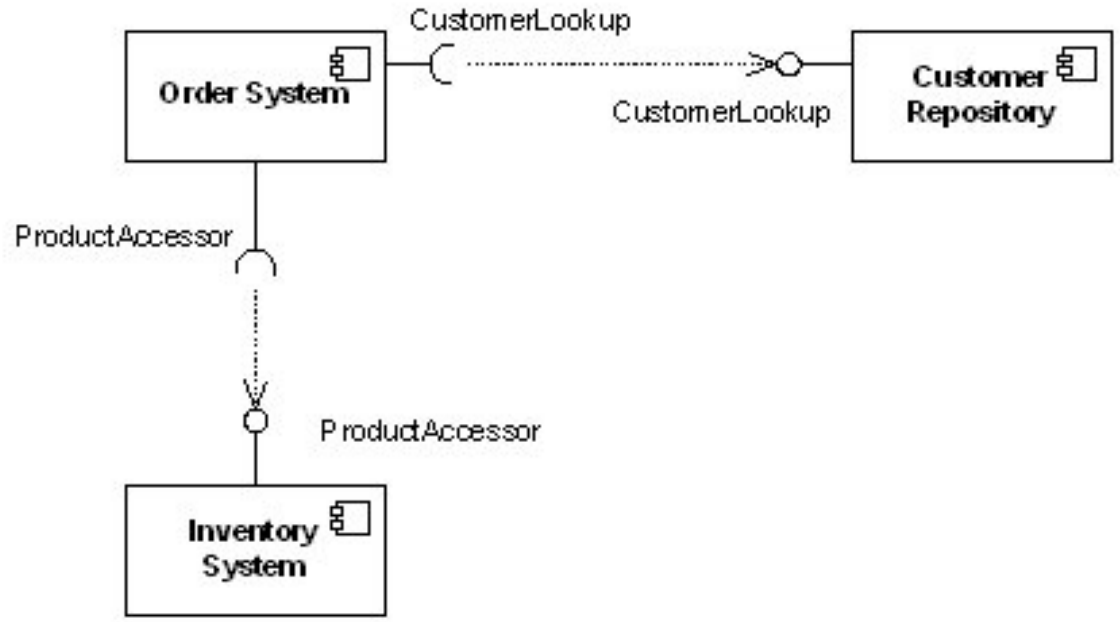
- Component Diagrams contain mainly *components*, *interfaces* and *relationships*.
- Components are related via dependency relationships.
- Run time components show the mapping of classes to run time libraries such as:
  - Java applets
  - Active-X components
  - DLLs

Component diagram



(UML 1.X notation)

Component diagram



A Component diagram (UML 2.0) that shows how the Order System component depends on other components

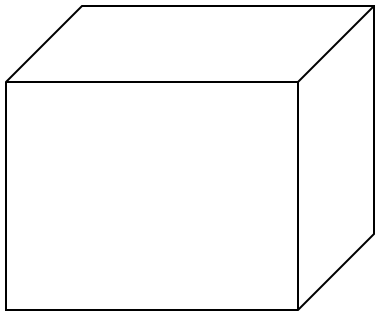
Deployment Diagram

- Shows the physical architecture of a computer-based system.
- It can show
  - the computers and devices,
  - their connections with one another
  - the software that sits on each machine
- Maps the software architecture to the hardware architecture.

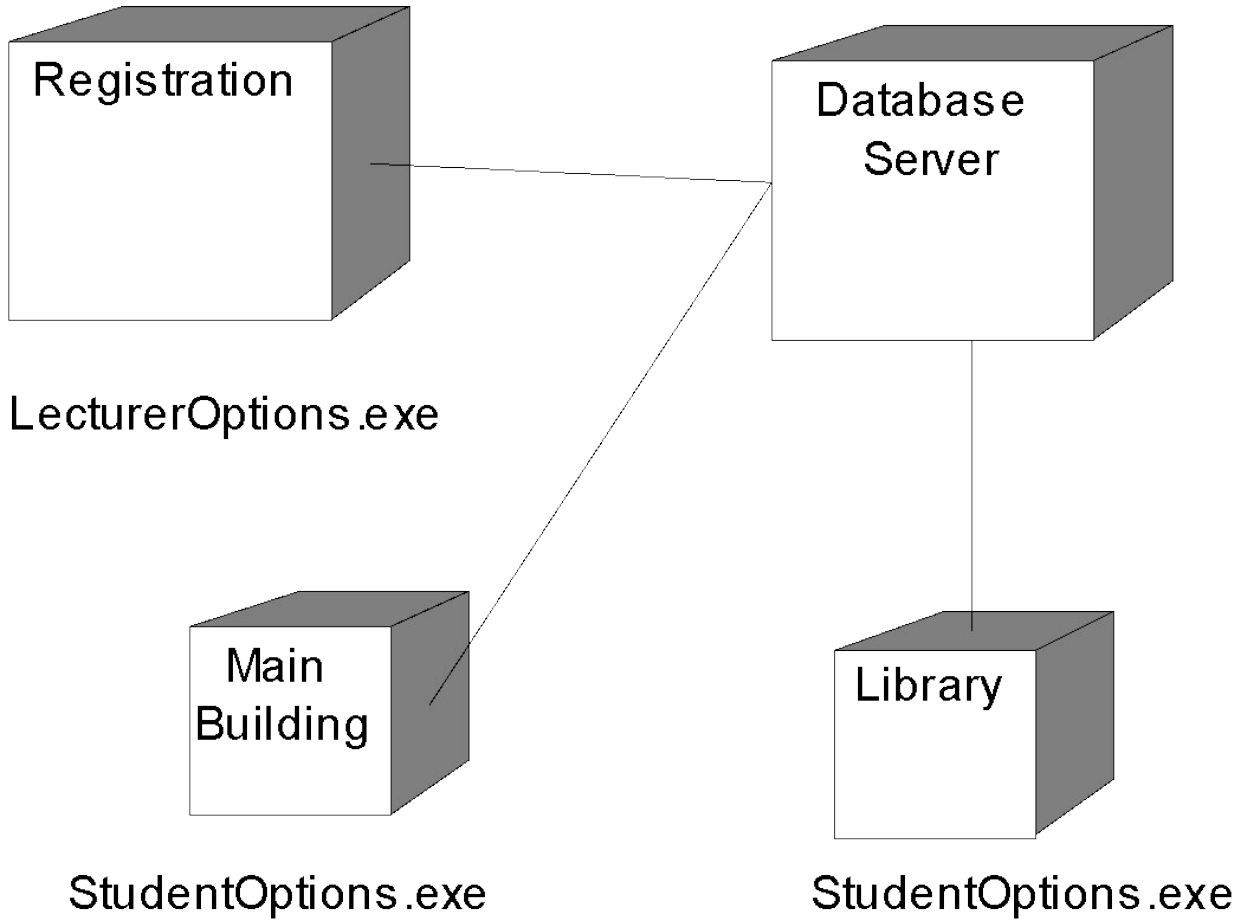
Deployment View

- Used **for distributed systems only**.
- The main hardware item is a *node*, a generic name for any kind of computing resource.

UML node icon



Deployment Diagram



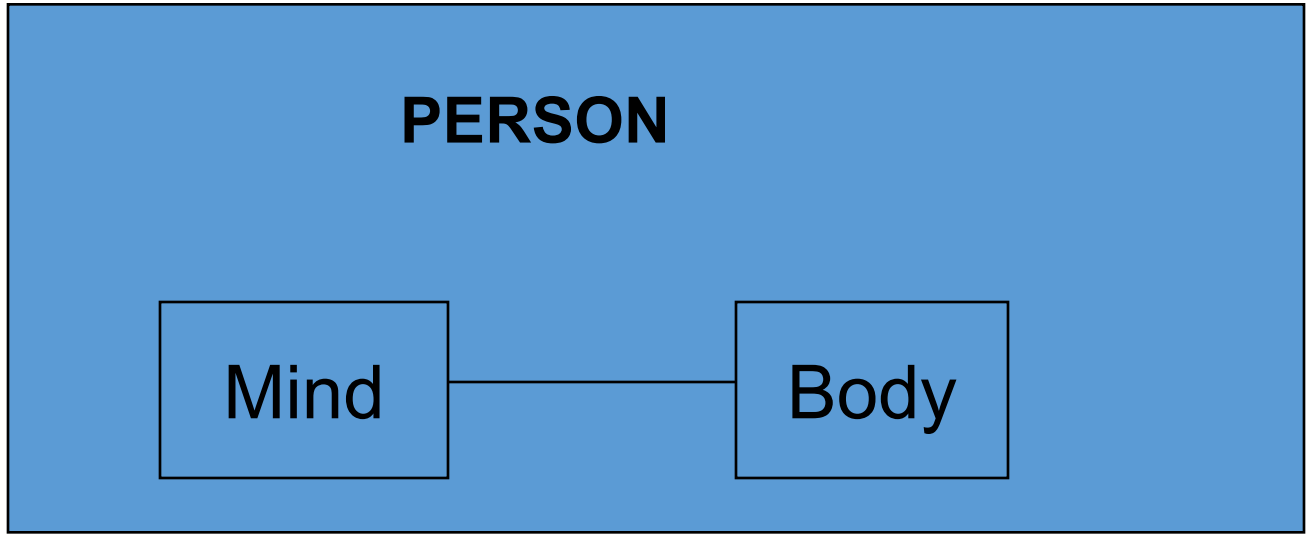
What is New in UML 2.0

- **New Diagrams**

  Composite Structure Diagram

  - **Show classes internal structure**

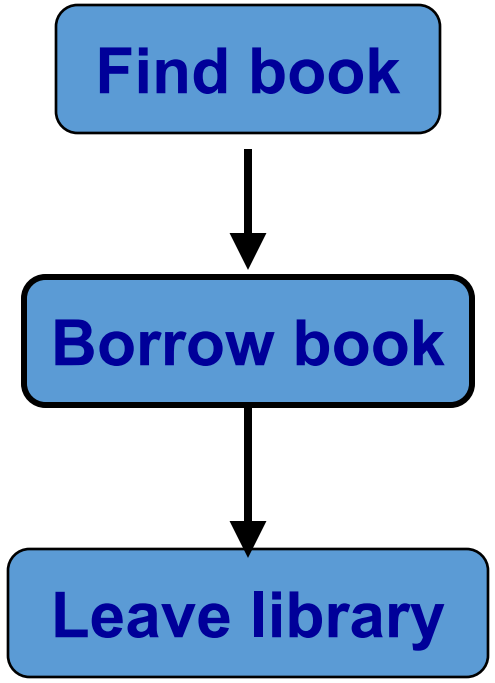
eg.



Interaction Overview Diagram

- Expands the Activity Diagram

eg. Consider three activities in visiting a library



Interaction Overview Diagram

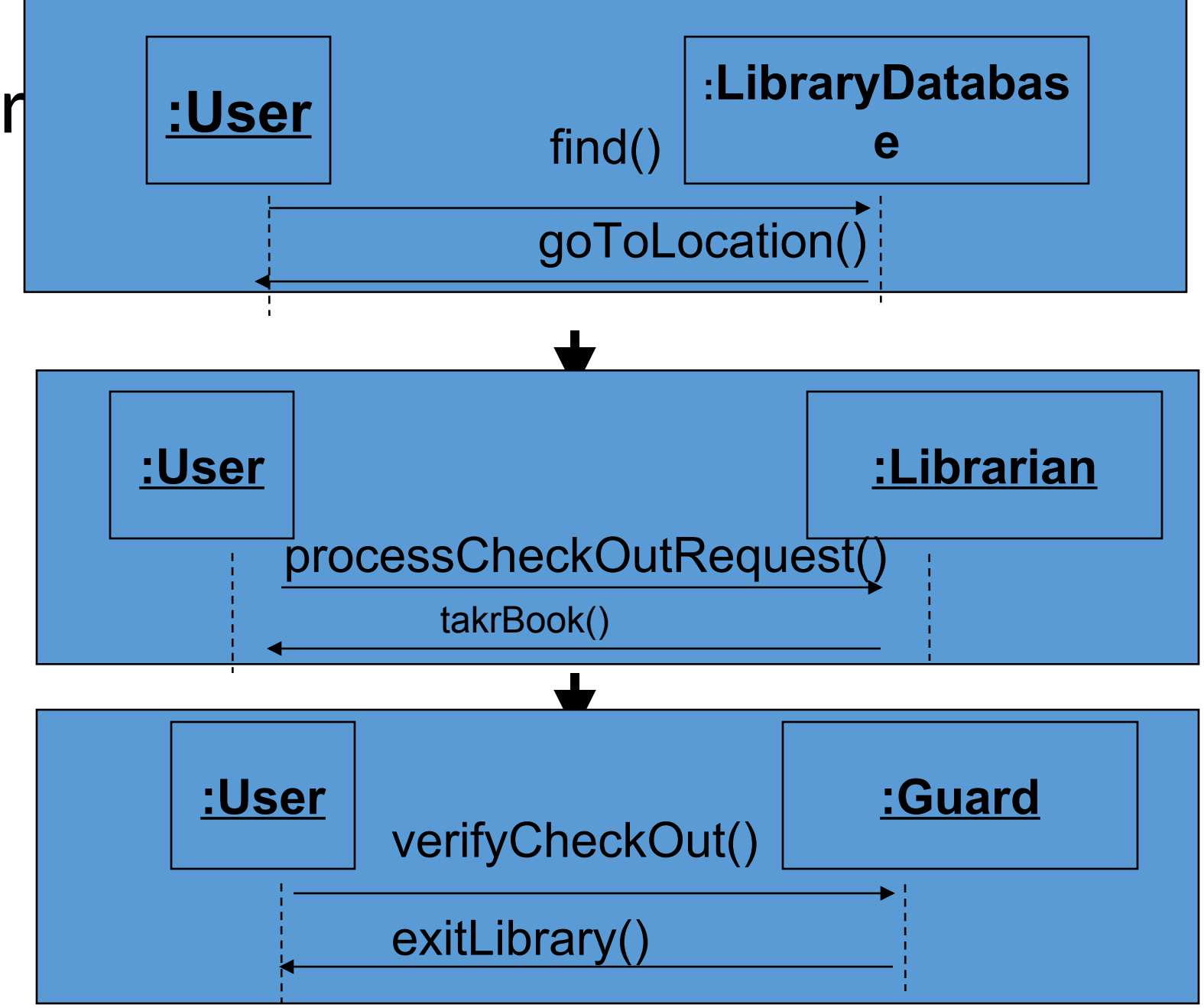
- Lets analyze each activity

Find Book : Ask library database to locate the book. Database responds by telling you to go to the books location.

Borrow Book : Ask the librarian to check the book out to you. After checkout, the librarian tells you to take the book.

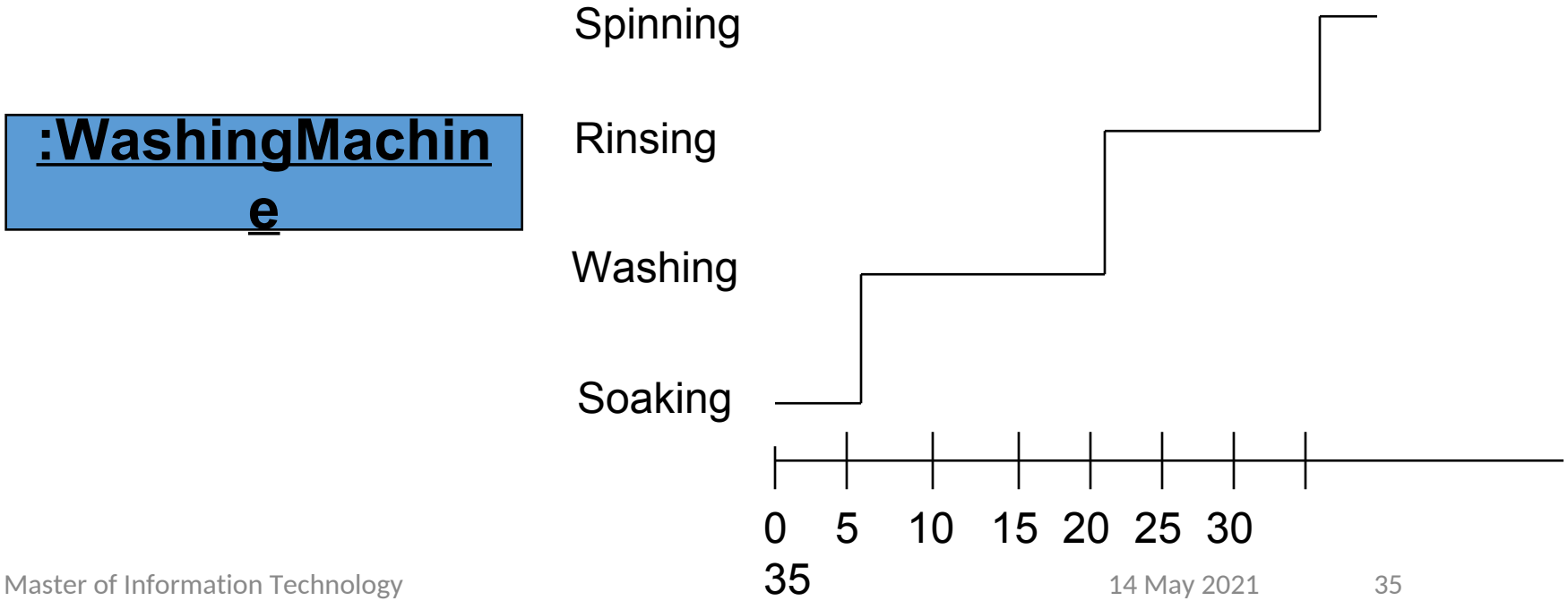
Leave Library : You can leave the library only if a guard verifies that you have checked out the book.

Inter


:User
:LibraryDatabase
find()
goToLocation()
:User
:Librarian
processCheckOutRequest()
takrBook()
:User
:Guard
verifyCheckOut()
exitLibrary()

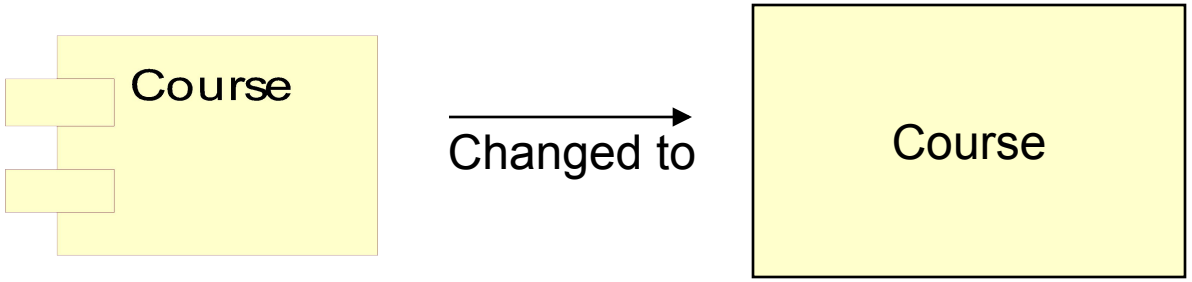
Timing Diagram

- Design to show how long an object is in a state.
- Sequence diagrams does not show the durations explicitly.



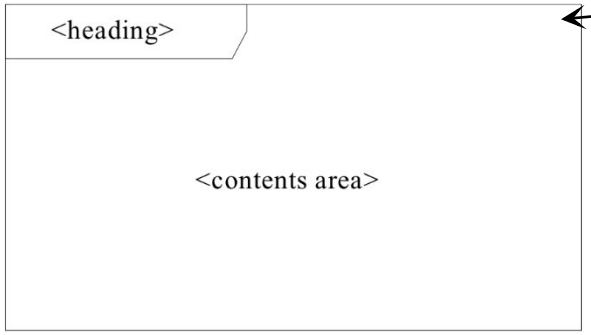
What is New in UML 2.0 cont..

- Rename collaboration diagram to **communication** diagram.
- Component symbol



Notation for all UML 2 Diagrams

- Mandatory : contents area
- Optional: The contents area can be surrounded with frame and a heading



Fram
e

<heading> ::= [<diagram kind>]<name>[<parameters>]

Diagram Kinds

Activity diagram
Class diagram
Component diagram
Interaction diagram
Package
state machine diagram
Use case diagram

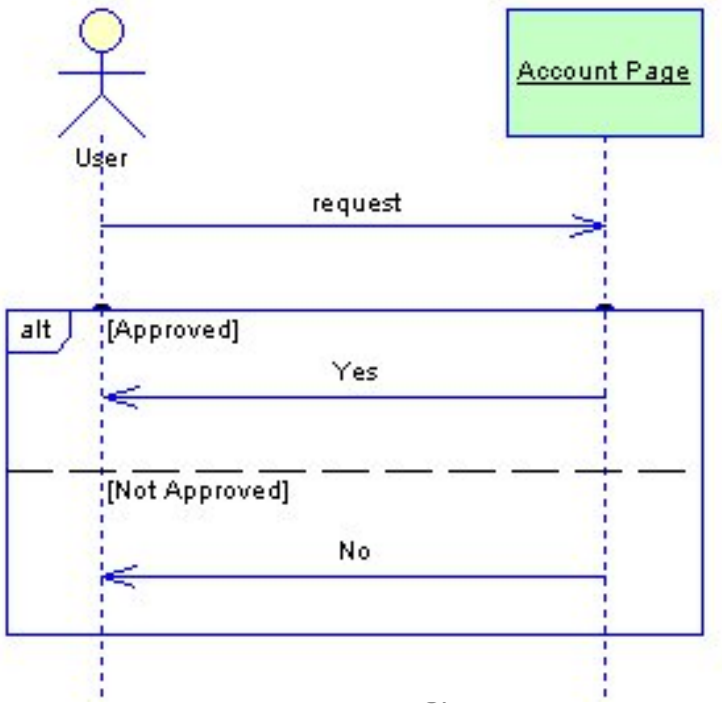
The following abbreviations are usually used for diagram frames:

- **act** (for activity frames)
- **cmp** (for component frames)
- **sd** (for interaction frames)
- **pkg** (for package frames)
- **stm** (for state machine frames)
- **uc** (for use case frames)

Why are interaction frames abbreviated with “sd”?
sequence diagram

Interaction Fragment

- Interaction Fragment
  - An interaction fragment allows you to group related messages in a sequence diagram.



The User sends a request to the Account Page.
The two alternative responses and the conditions on which they depend, are contained within an interaction fragment.

Combined Fragment

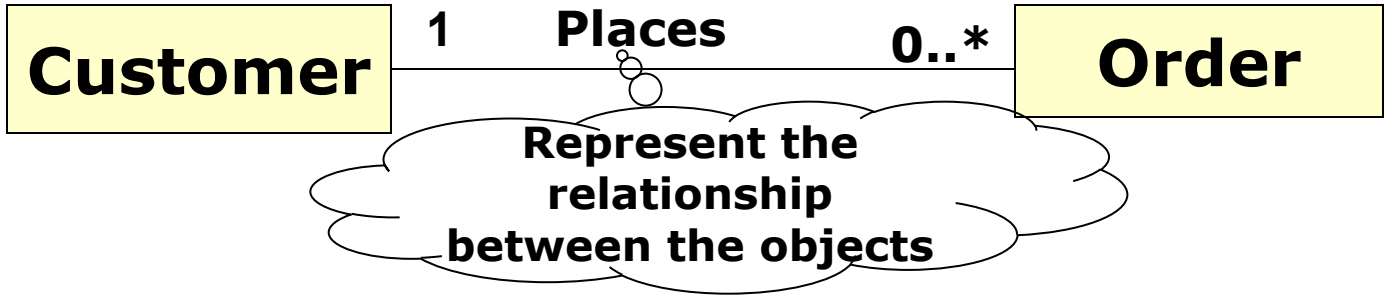
- Is a subtype of interaction fragment
- defines an expression of interaction fragments
- defined by an **interaction operator** and corresponding **interaction operands**

- Association

  e.g.

  A CUSTOMER PLACES zero or more ORDERS

  An ORDER IS PLACED BY one and only one CUSTOMER



- Multiplicity
  - The minimum and maximum number of occurrences of one object class for a single occurrence of the related object class.

    e.g. Exactly one -> **1**

    Zero or 1 -> **0..1**

    Zero or more -> **0..*** or ***

    1 or more -> **1..***

    Specific range -> **7..9**

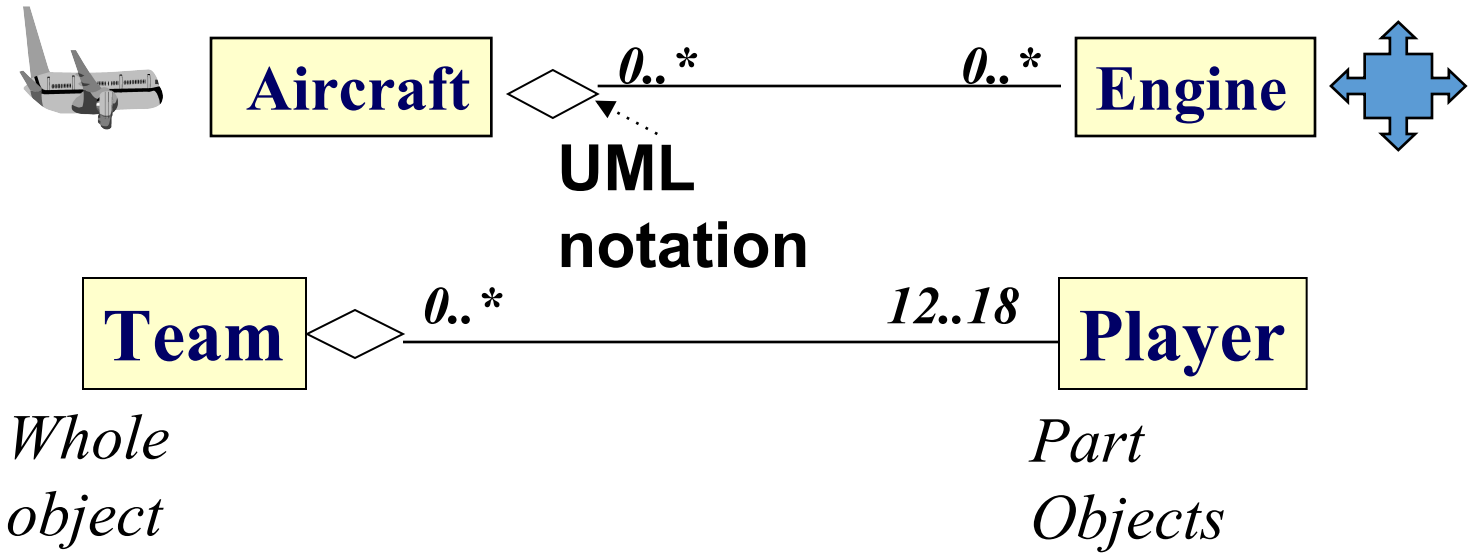
- *Aggregation*
  - A relationship in which one larger “whole” class contains one or more smaller “parts” classes. Conversely, a smaller “part” class is part of a “whole” larger class.

    e.g. *A club* – a club is made up of several club members

    A computer – a computer contains a case, CPU, motherboard, power supply …etc.

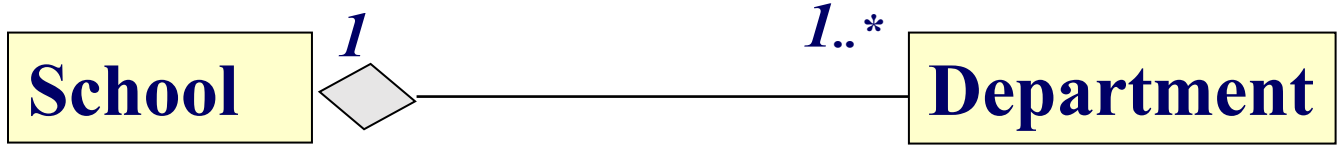
- *Aggregation* ***(Removed in UML 2.x)***

  some more examples…



- ***Composition***
  - An aggregation relationship in which the “whole” is responsible for the creation and destruction of its “parts”.
  - If the “whole” were to die, the “part” would die with it.
  - **A stronger form of aggregation.**
    - The relationship between club and club member would not be composition, because members have a life out-side the club and can, belong to multiple clubs.

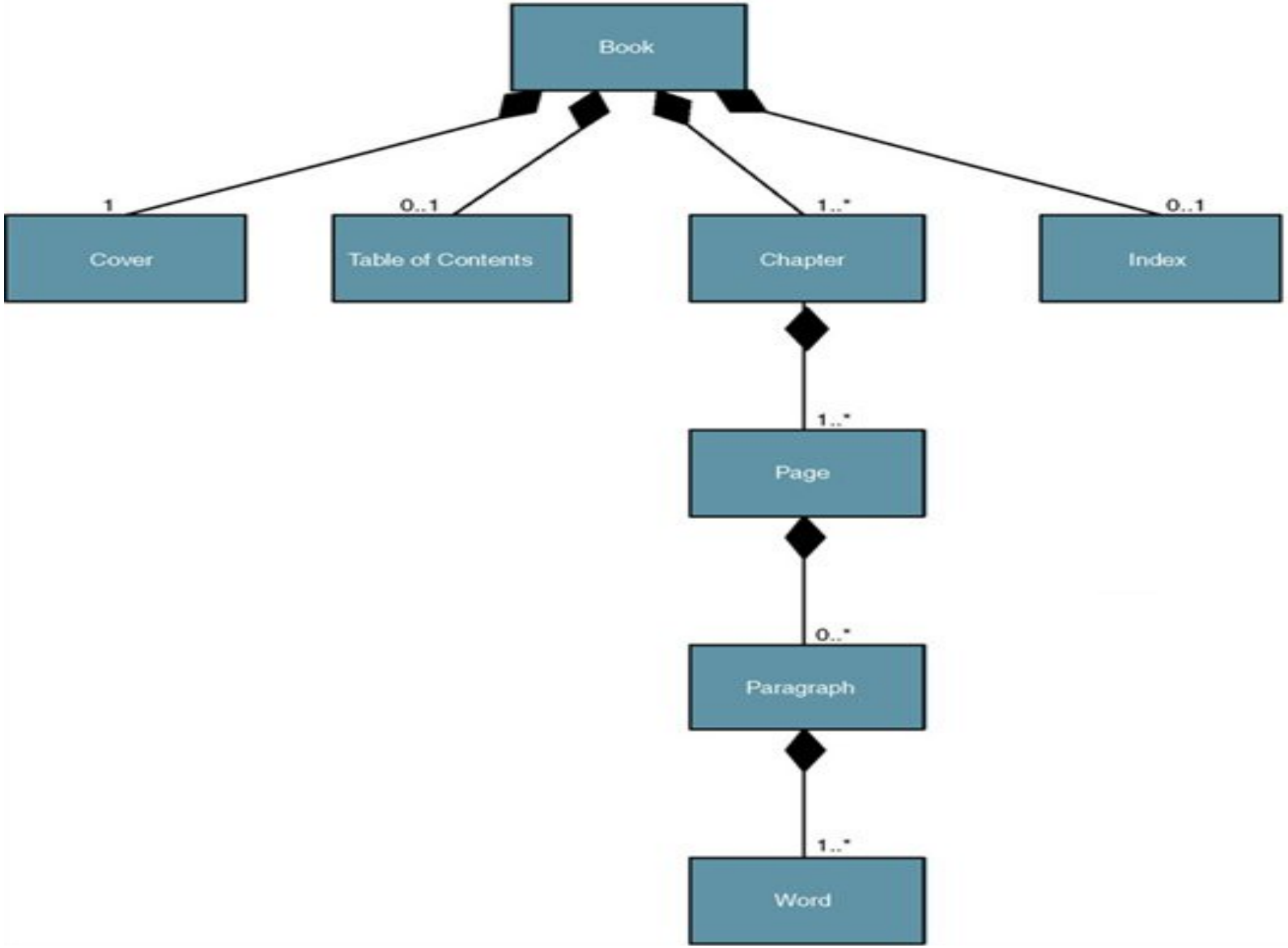
- ***Composition***
  - Drawn with a filled diamond.



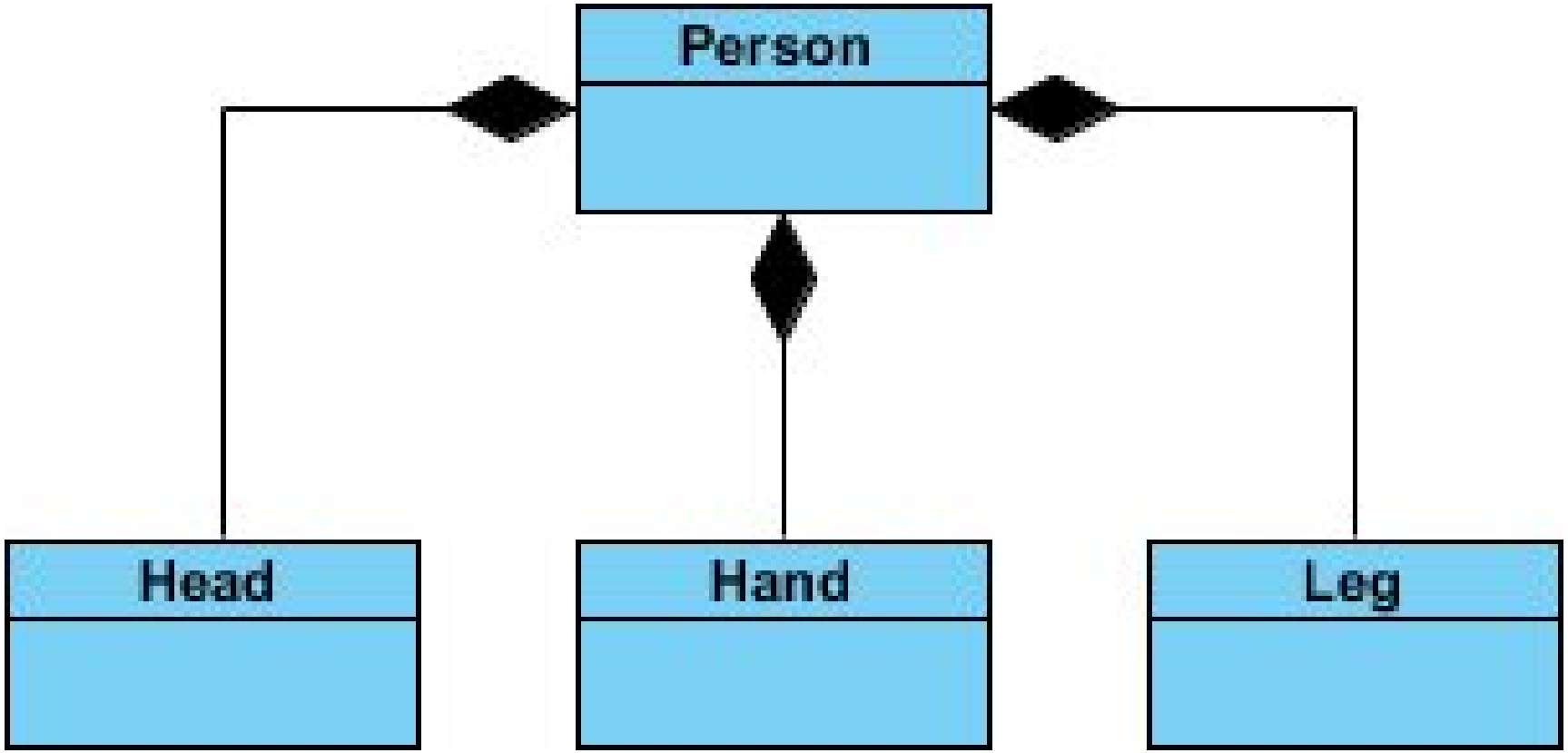
Each “part” can belong to only one “whole”, therefore, multiplicity needs to be specified only one for the “whole”

Components will live and die with the whole object

Composition



Another Example of Composition



- Abstraction
  - A mechanism to reduce and filter out details so that one can focus on a few concepts at a time. Focus on Essentials
  - Filter out an object's properties and operations until just the ones you need are left.

  Ignore the irrelevant. Ignore the unimportant.

  eg. Book – ~~Bookshop you bought~~

- Inheritance
  - The **concept** wherein methods and/or attributes defined in a class can be inherited or reused by another class.

    e.g. some individuals in the room might be classified as STUDENTS and TEACHERS. Thus, STUDENT and TEACHER classes are members of the class PERSON

- Generalization / Specialization
  - A **technique** wherein the attributes and behaviors that are common to several types of classes are grouped / abstracted into their own class called a supertype.
  - The attributes and methods of the supertype class are then inherited by those classes (subtype)
  - Sometimes abbreviated as gen/spec.

Generalization

Person
firstName lastName birthdate gender
walk jump talk sleep

Inheritable
<u>Attributes</u>
And
<u>behavior</u>

Specialization

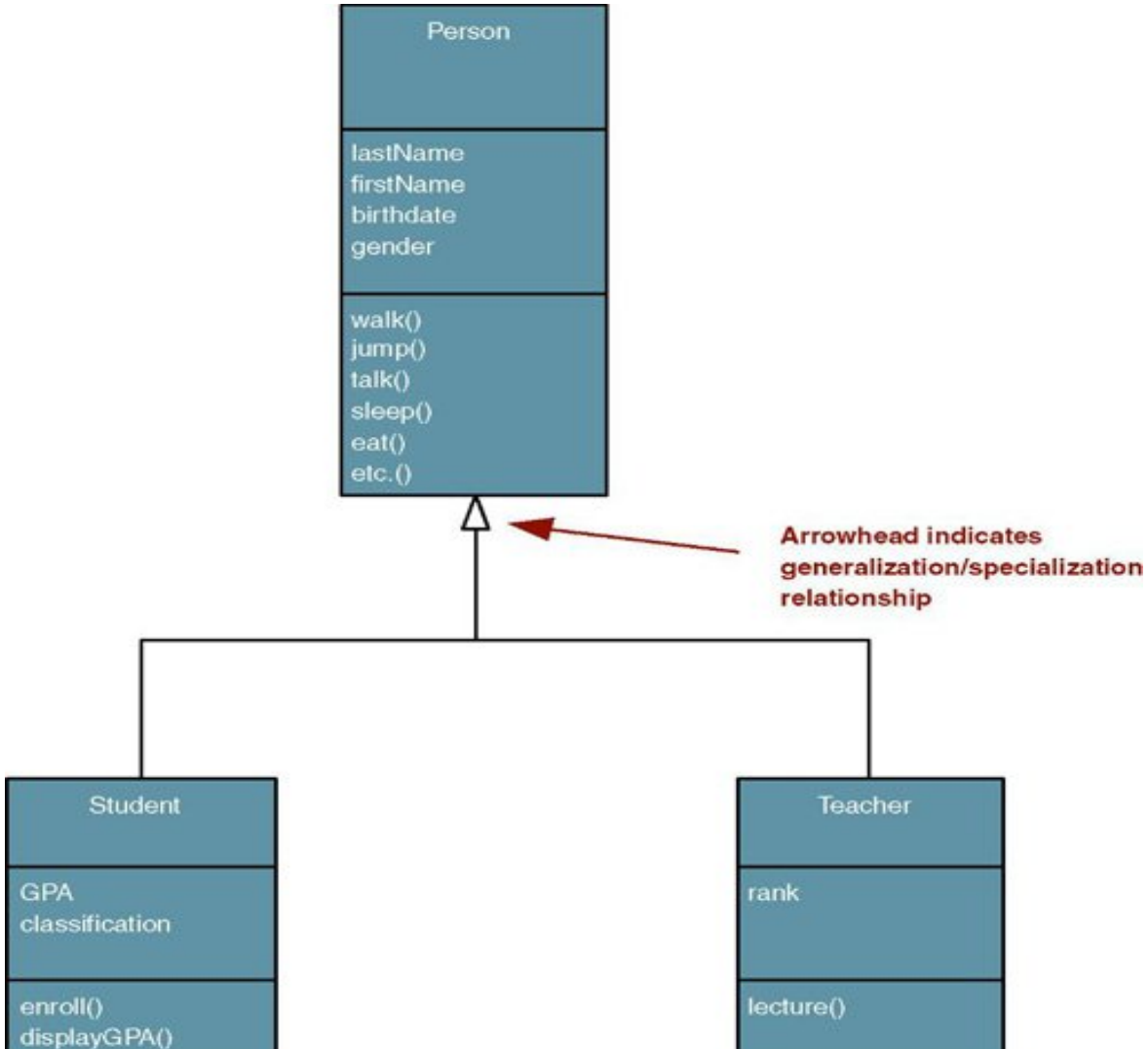
Student
GPA Classificatio n
enroll displayGPA

Teacher
rank
lecture

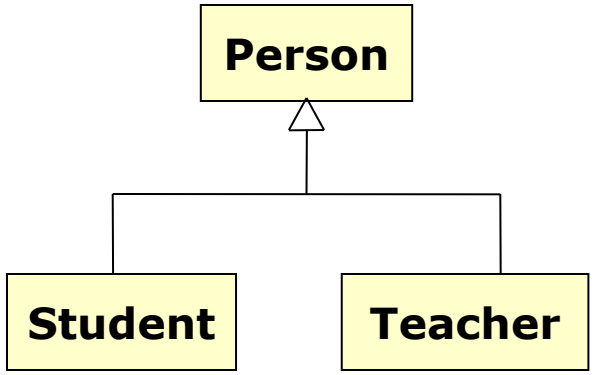
+

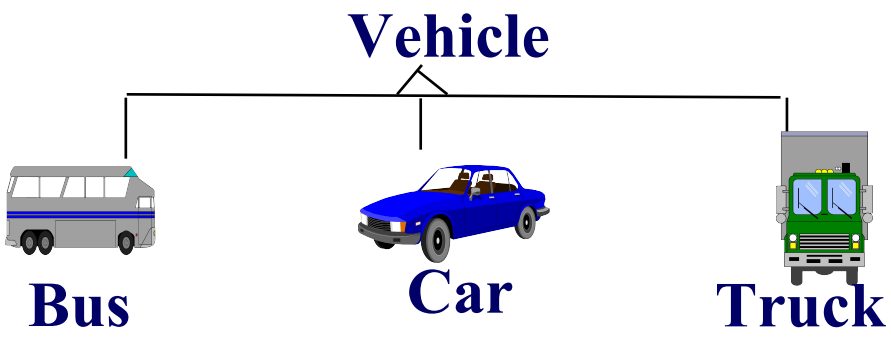
firstName
lastName
birthdate
gender
walk
jump
talk
sleep

UML Representation of Generalization/Specialization



- Generalization / Specialization





*** Specialized classes inherits from the parent class**

Polymorphism

- the concept that different objects can respond to the same message in different ways.
  - It allows different forms of the same service to be defined.
  - Sometimes an operation has the same name in different classes.

    eg. You can open a door, you can open a window, or a bank account. In each case, you are performing a different operation.

- **Polymorphism**
  - In object-orientation, each class “knows” how that operation is supposed to take place.
  - This is polymorphism.
  - *Related terms used in OO programming language*

    Function Overloading,

    Operator Overloading,

    Method Overriding

System concepts for object modeling

- **Polymorphism -** Function(Method) Overloading

In some programming languages, **function overloading** or **method overloading** is the ability to create
multiple functions of the same name
with different implementations.

Same Function Name, Different arguments & Functionality

```
void myFunction()
void myFunction(int a)
void myFunction(float a)
void myFunction(int a, float b)
float myFunction (float a, int b)
```

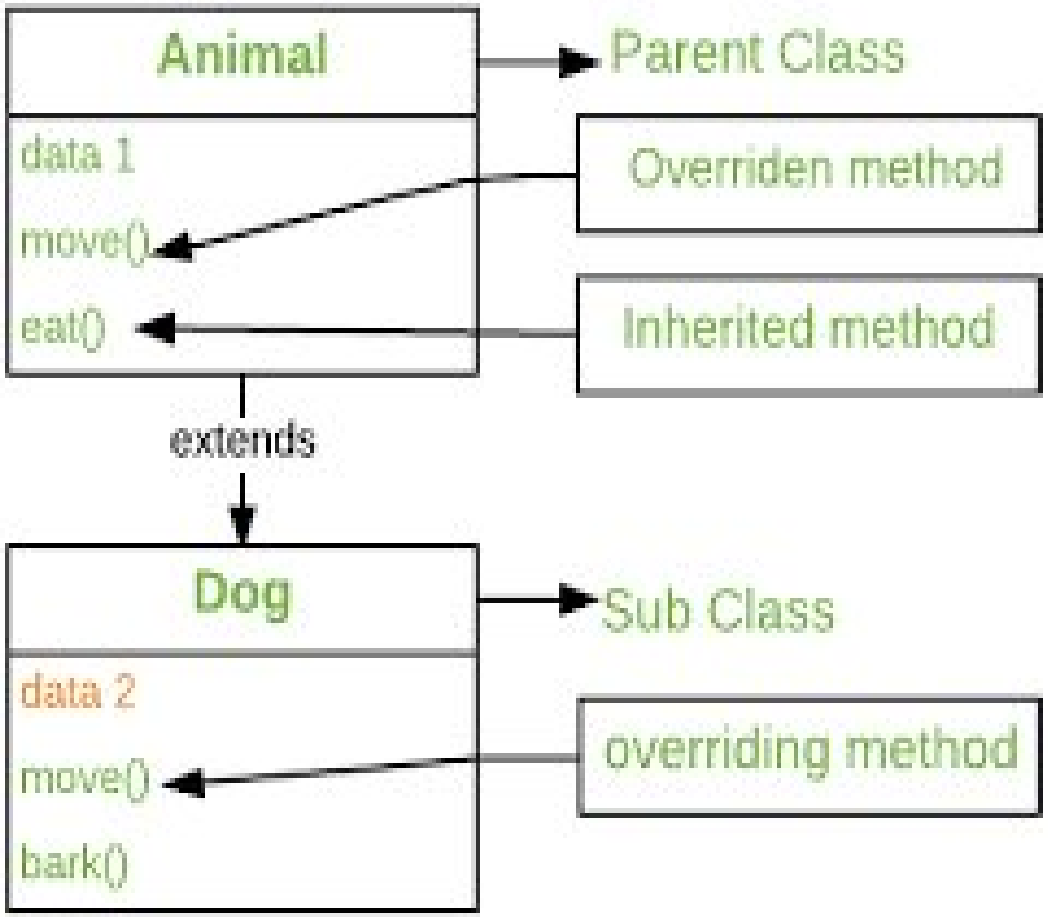
Method Overloading

```
int add(int a, int b) {
   return a + b;
}

int add(int a, int b, int c) {
   return a + b + c;
}

double add(double a, double b) {
   return  a  +  b ;
}
```

add(3.8, 6.5) ;

Polymorphism

Overriding – a technique whereby a subclass (subtype) uses an attribute or behavior of its own instead of an attribute or behavior inherited from the class (supertype).



Polymorphism – Operator Overloading

- In languages like C++, we can change the way operators work for user-defined types like objects and structures. This is known as operator overloading.
- Suppose we have created three objects **c1**, **c2** and **result** from a class named **Complex** that represents complex numbers.
- we can redefine how the + operator works and use it to add the complex numbers of c1 and c2 by writing the following code:

  result = c1 + c2;

  instead of something like

  result = c1.addNumbers(c2);

Operator Overloading in C++

```
class complex
 {
  float real,imag;
   public : complex operator+(complex&);
            complex() { } //default constructor
            complex(float r, float s) // constructor
               { real=r; imag=s;}
            void printcomplex( )
               {
              std::cout<<"Real component="<<real<<'\n';
             std::cout<<"Imaginary component="<<imag;
               }
 };
```

complex operator+(complex&);

- Definition of the operator is like a member function, except the operator symbol is prefixed by the keyword operator.

```
complex complex::operator+(complex &a)
{
   complex z;
   z.real=real+a.real;
   z.imag=imag+a.imag;
   return z;
   }
```

```
int main()
  {
   complex var1(1.2,3.4), var2(4.5,6.5), outc;
   outc=var1+var2;
   outc.printcomplex();
   }
```

Sample Output :

Real component = 5.7

Imaginary component = 9.9

Q & A